

(12) **United States Patent**
Yu et al.

(10) **Patent No.:** **US 12,406,683 B2**
(45) **Date of Patent:** **Sep. 2, 2025**

(54) **KALMANNET: A LEARNABLE KALMAN FILTER FOR ACOUSTIC ECHO CANCELLATION**

(71) Applicant: **TENCENT AMERICA LLC**, Palo Alto, CA (US)

(72) Inventors: **Meng Yu**, Palo Alto, CA (US); **Hao Zhang**, Palo Alto, CA (US); **Dong Yu**, Palo Alto, CA (US)

(73) Assignee: **TENCENT AMERICA LLC**, Palo Alto, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 165 days.

(21) Appl. No.: **18/327,447**

(22) Filed: **Jun. 1, 2023**

(65) **Prior Publication Data**

US 2024/0404541 A1 Dec. 5, 2024

(51) **Int. Cl.**
G10L 21/0216 (2013.01)
G10L 21/0208 (2013.01)

(52) **U.S. Cl.**
CPC **G10L 21/0216** (2013.01);
G10L 2021/02082 (2013.01); **G10L 2021/02163** (2013.01)

(58) **Field of Classification Search**
CPC G10L 21/02; G10L 2021/02082; G10L 21/0216
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

11,727,912 B1 * 8/2023 Rao G10L 21/0208 381/66
2009/0281805 A1 * 11/2009 LeBlanc G10L 21/0208 704/E15.039
2020/0312346 A1 * 10/2020 Fazeli H04R 3/04
2021/0390975 A1 * 12/2021 Wang G10L 25/93
2022/0277721 A1 * 9/2022 Zhang G10L 21/0208
2023/0094630 A1 * 3/2023 Zhang G06V 10/82 704/226

OTHER PUBLICATIONS

Revach, Guy, et al. "KalmanNet: Neural network aided Kalman filtering for partially known dynamics." IEEE Transactions on Signal Processing 70 (2022): 1532-1547. (Year: 2022).*
Fazel, Amin, Mostafa El-Khamy, and Jungwon Lee. "Deep Multitask Acoustic Echo Cancellation." Interspeech. 2019. (Year: 2019).*
Yu, Meng, et al. "NeuralEcho: A self-attentive recurrent neural network for unified acoustic echo suppression and speech enhancement." arXiv preprint arXiv:2205.10401 (2022). (Year: 2022).*
Ma, Lu, et al. "Acoustic echo cancellation by combining adaptive digital filter and recurrent neural network." arXiv preprint arXiv:2005.09237 (2020). (Year: 2020).*

* cited by examiner

Primary Examiner — Shaun Roberts

(74) Attorney, Agent, or Firm — Sughrue Mion, PLLC

(57) **ABSTRACT**

A method and apparatus comprising computer code configured to cause a processor or processors to receive an audio signal obtained from a microphone, input the audio signal into a neural-network based AEC model, and output an AEC signal from the neural-network based AEC model in which AEC is applied to the audio signal, and the AEC signal is a version of the audio signal in which acoustic echo noise of the audio signal is suppressed and target audio of the audio signal is sustained.

20 Claims, 12 Drawing Sheets

700

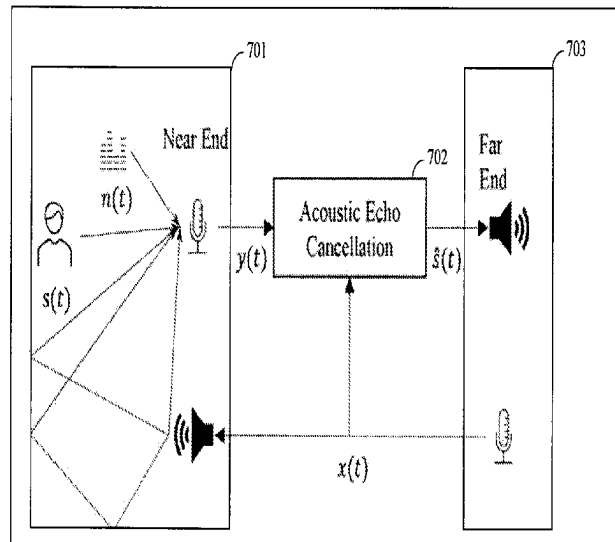


FIG. 1

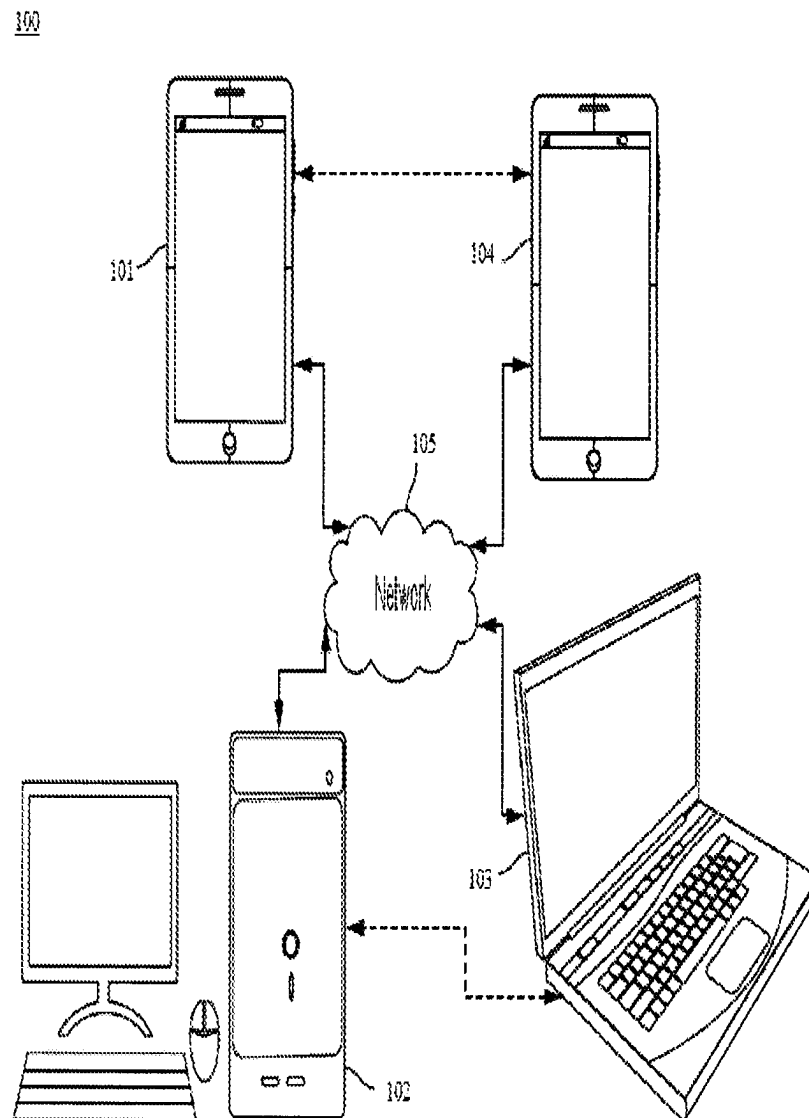


FIG. 2

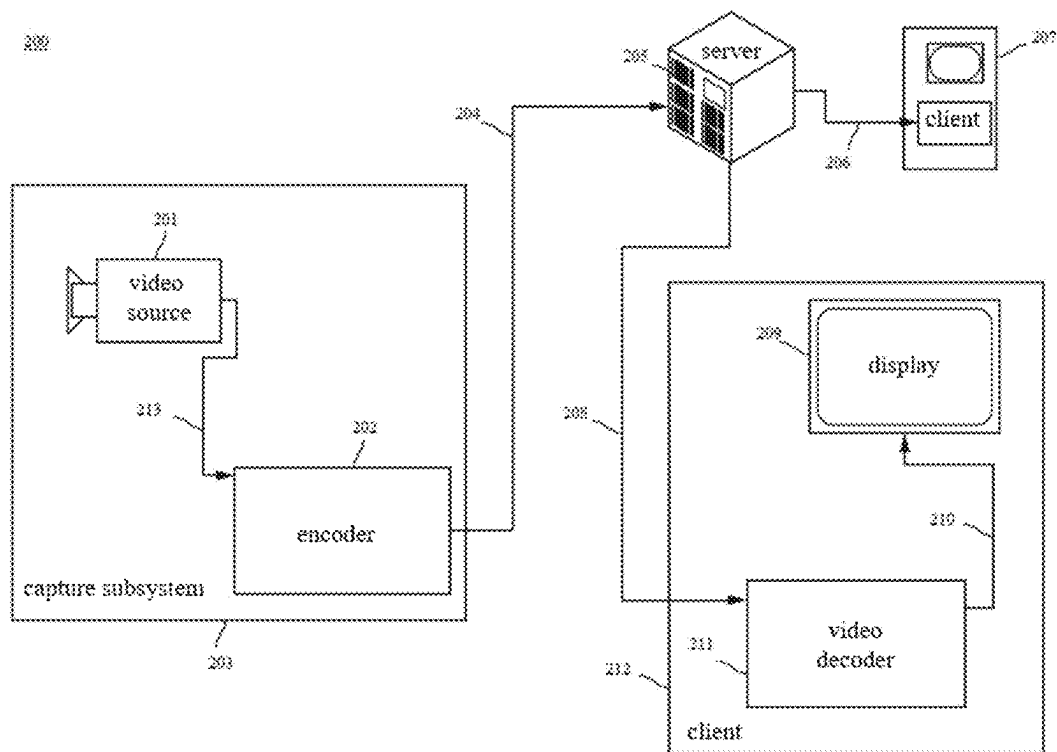


FIG. 3

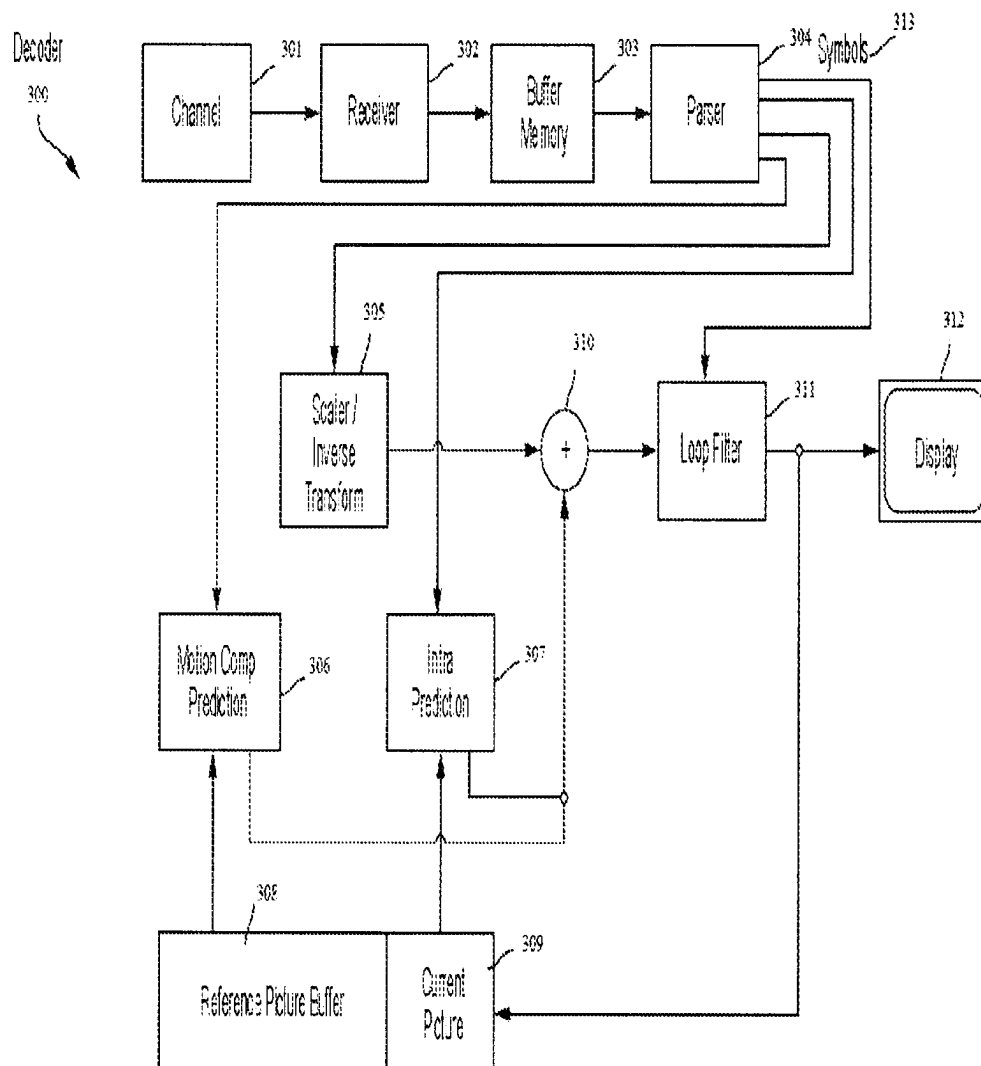


FIG. 4

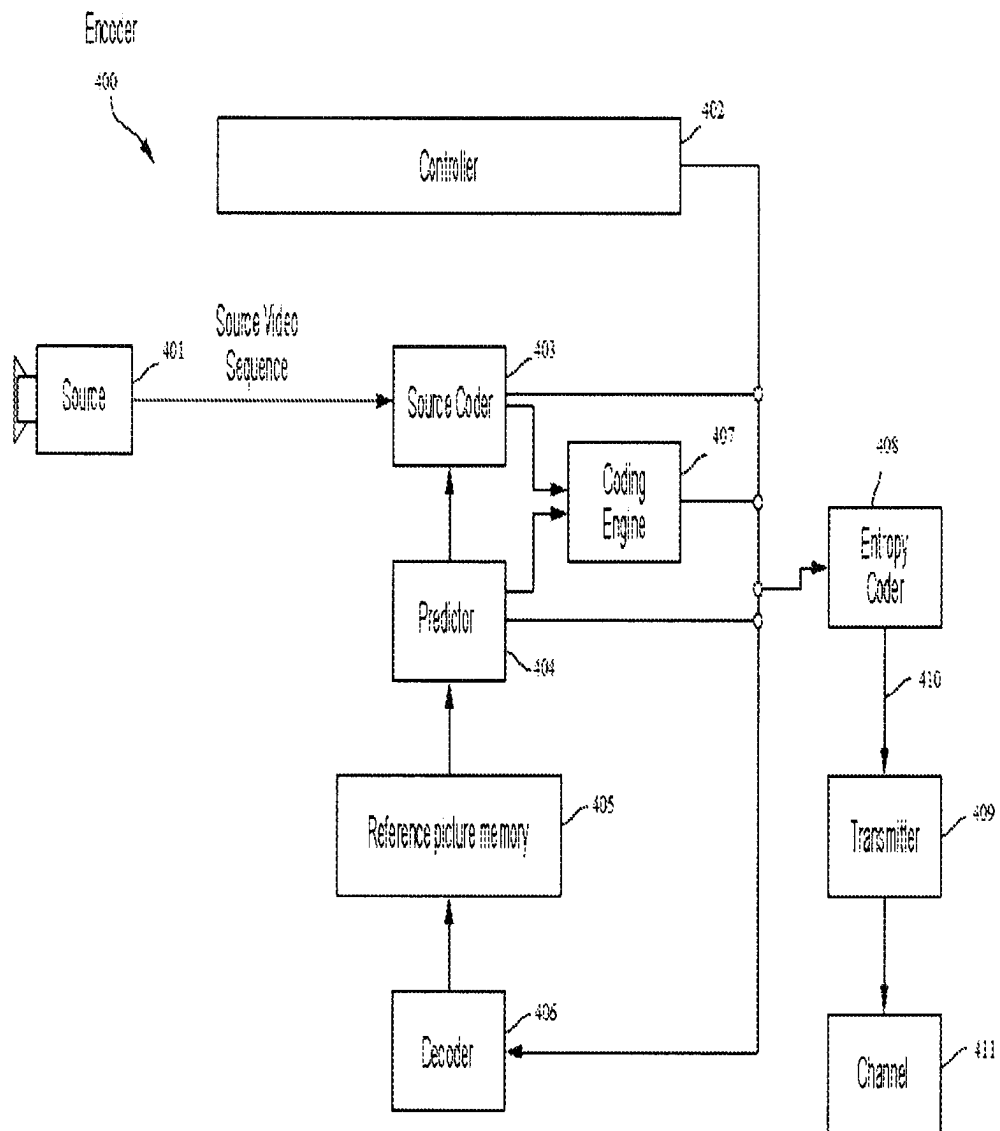


FIG. 5

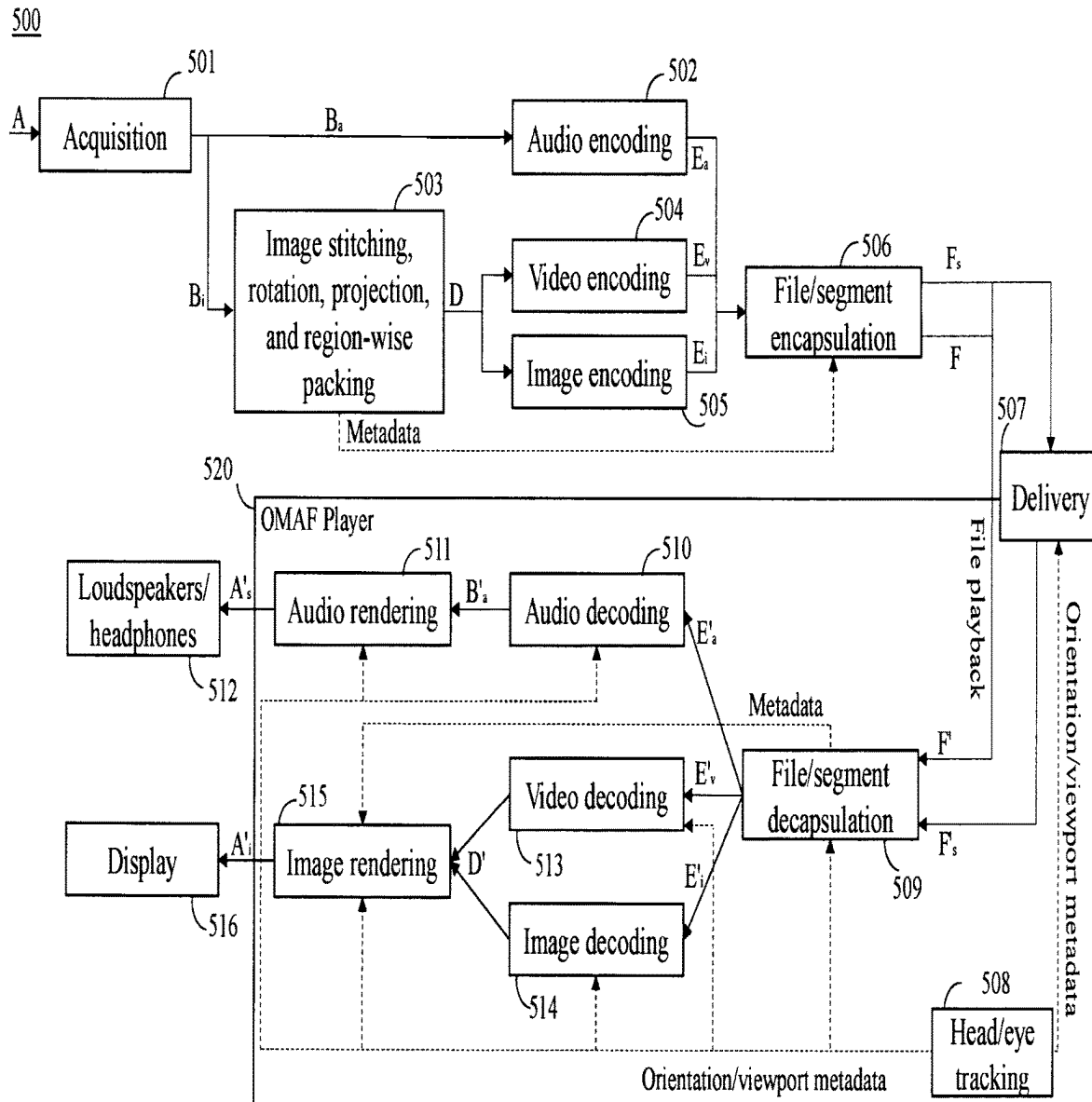


FIG. 6

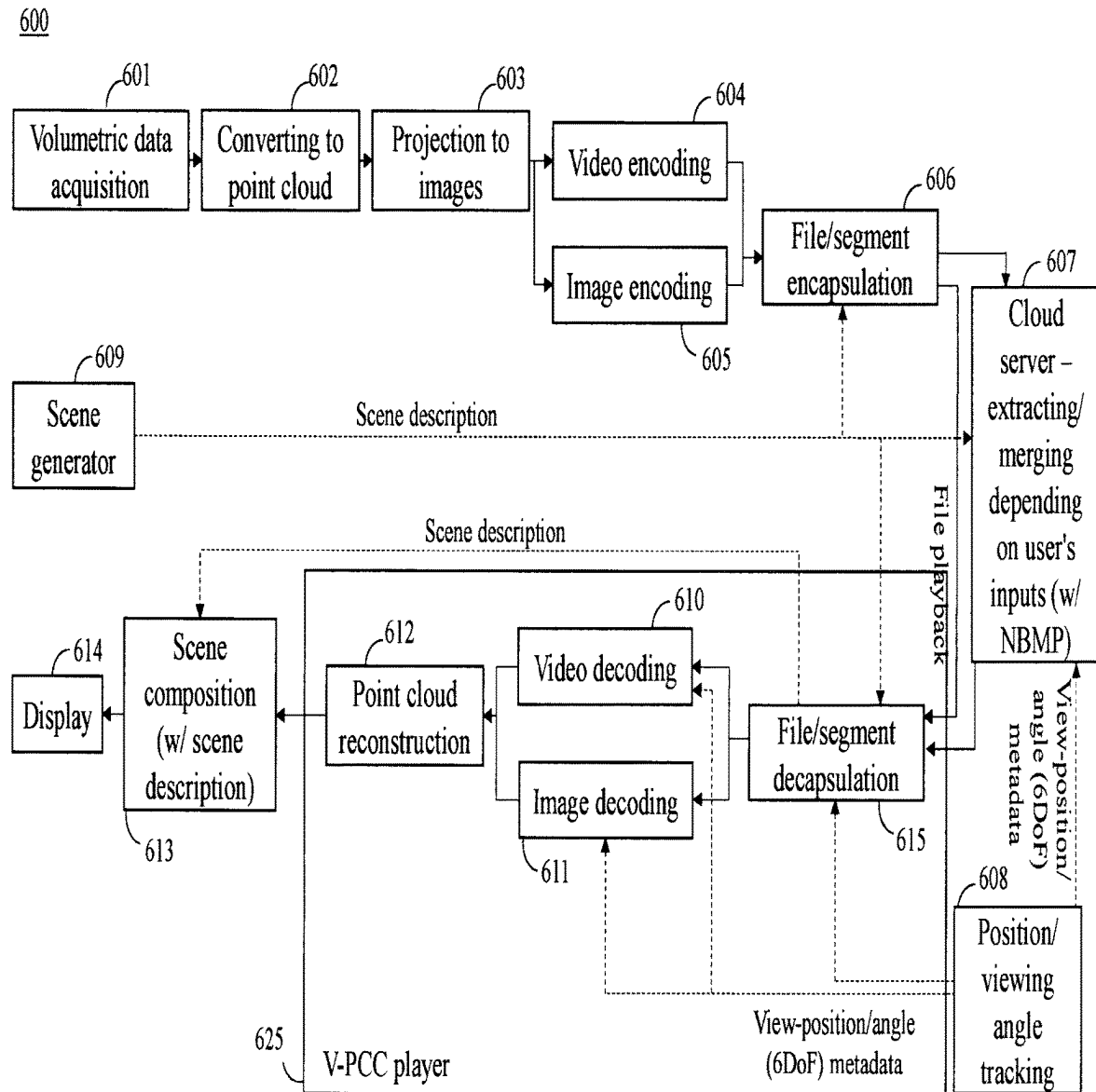


FIG. 7

700

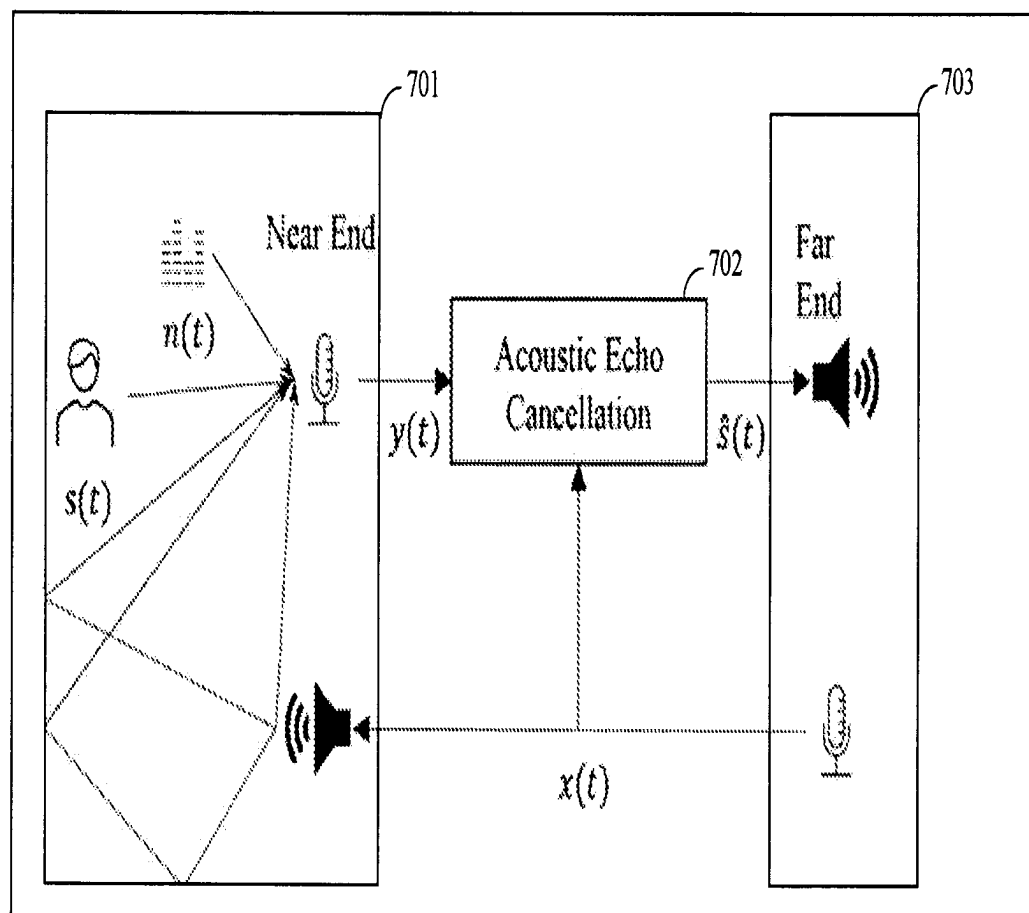


FIG. 8

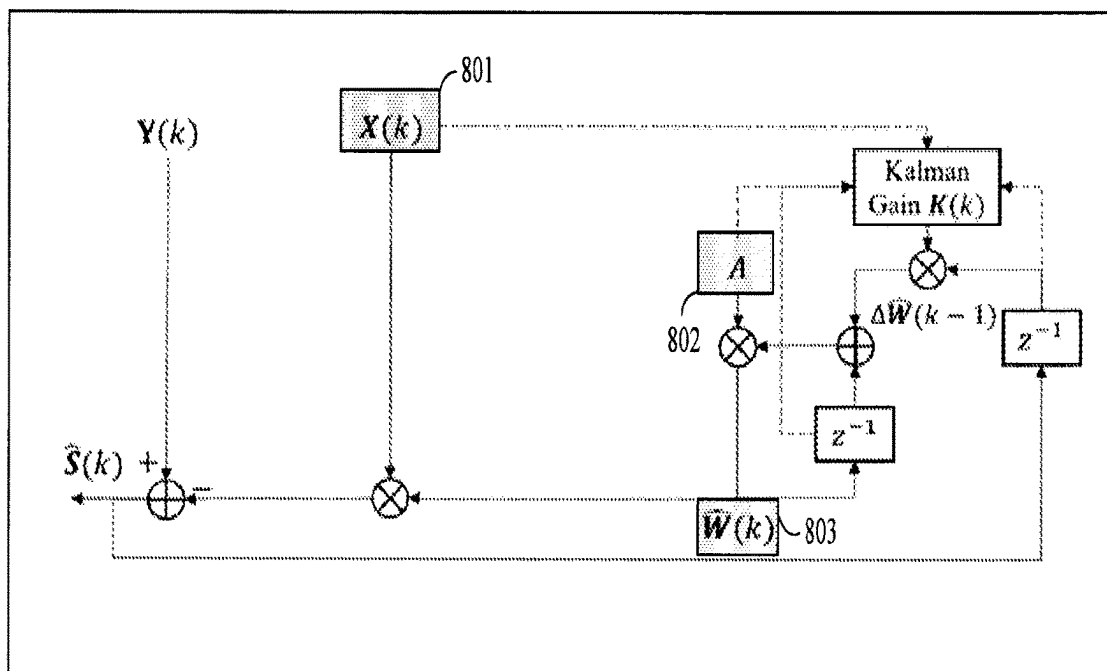
800

FIG. 9

900

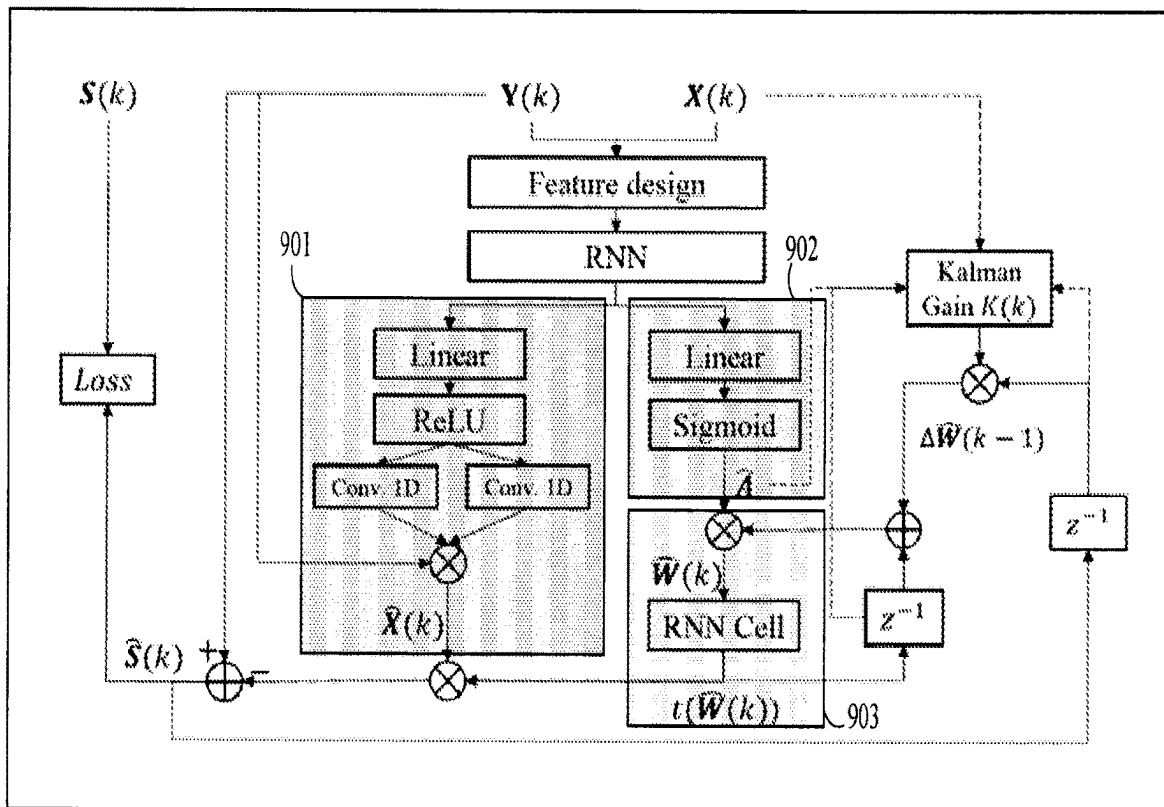


FIG. 10

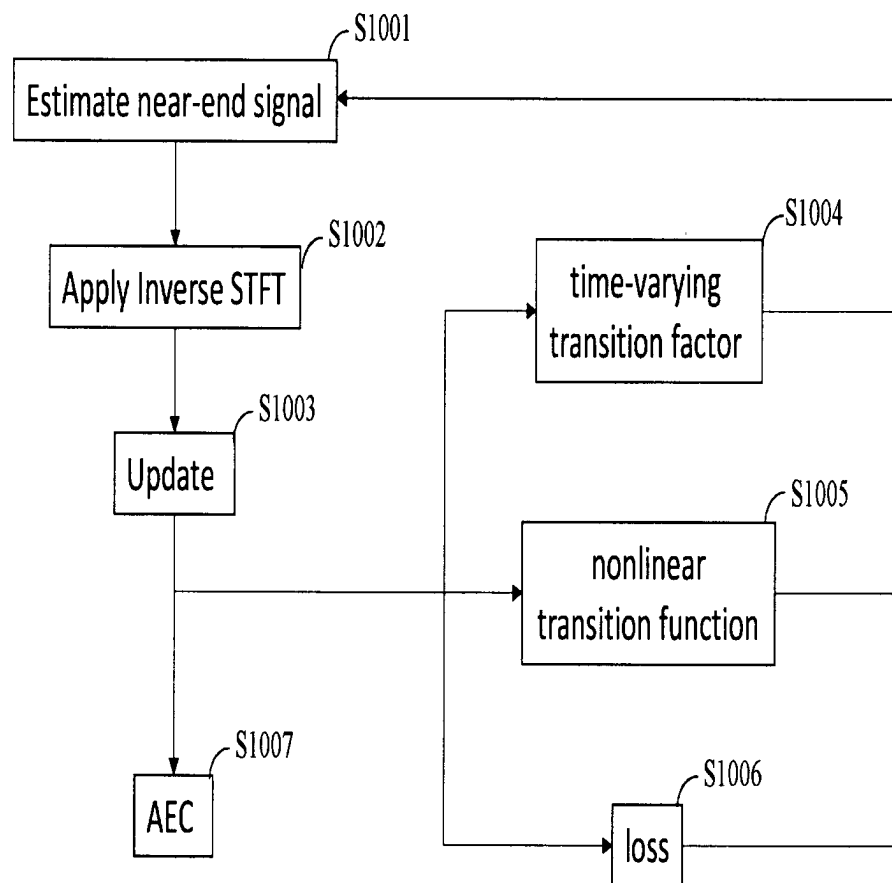
1000

FIG. 11

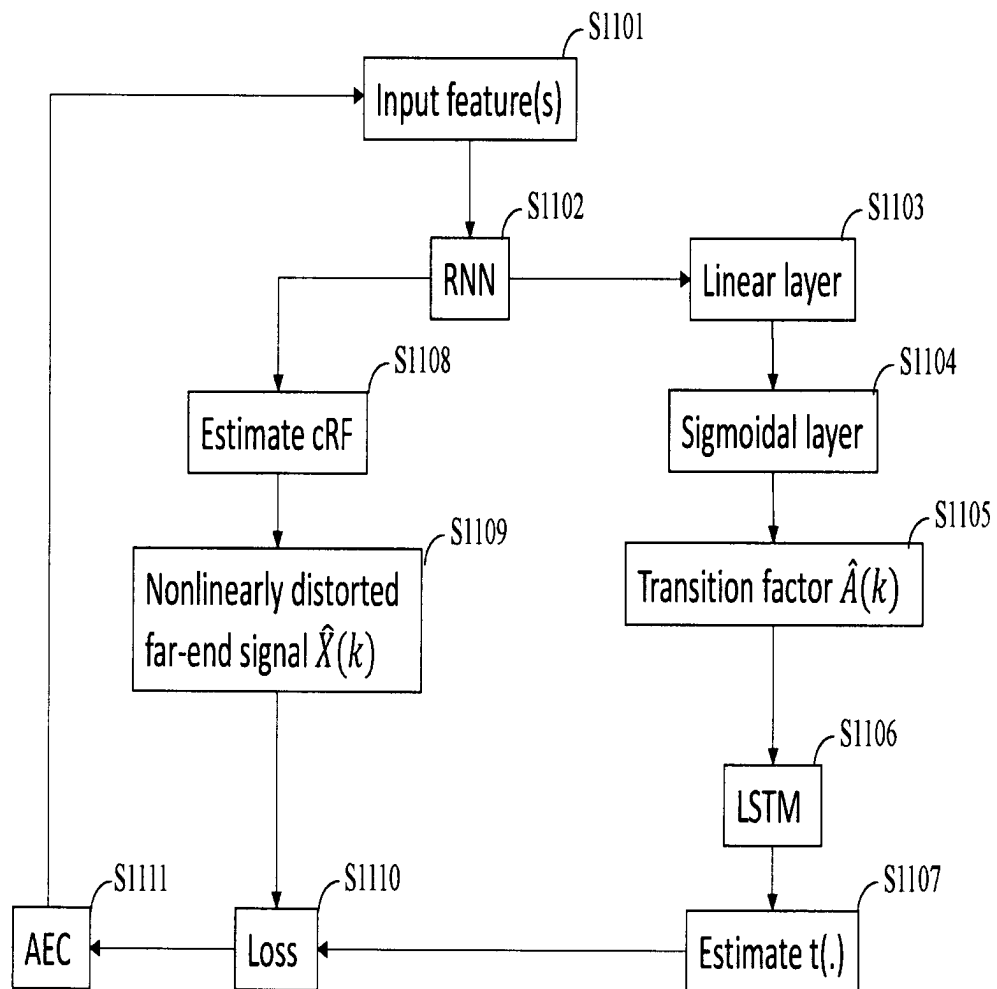
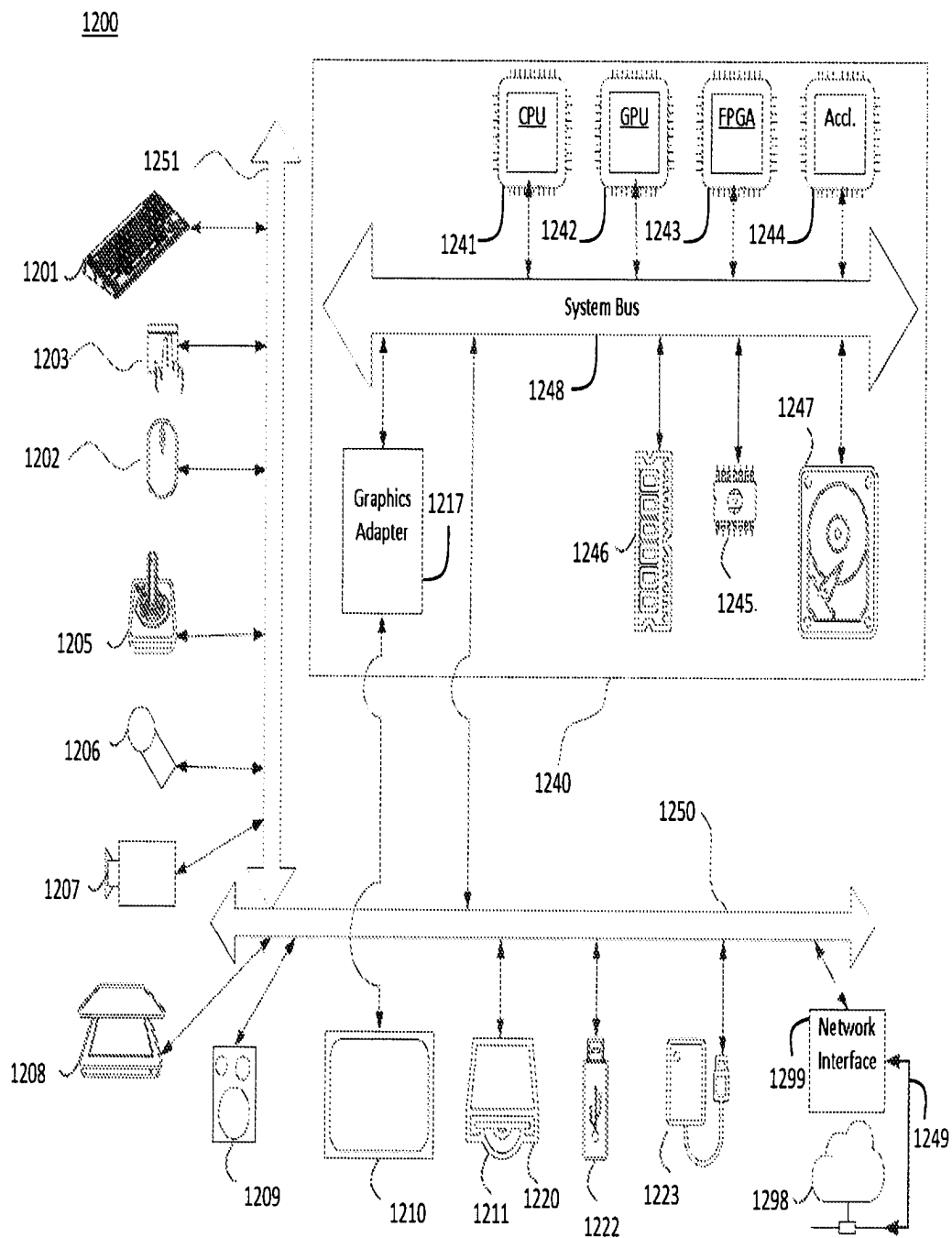
1100

FIG. 12



1

KALMANNET: A LEARNABLE KALMAN FILTER FOR ACOUSTIC ECHO CANCELLATION

BACKGROUND

1. Field

The present disclosure is directed a set of advanced audio technologies for addressing acoustic echo cancellation (AEC) problems due at least to their robustness to double-talk and fast convergence.

2. Description of Related Art

AEC, as an active and challenging research problem in the domain of speech processing, has been studied for decades and is widely used in mobile communication and teleconferencing systems. The goal of AEC is to eliminate the far-end signal from the near-end microphone signal so as to remove the echo of the far-end signal (back to the far end). In conventional digital signal processing (DSP) based adaptive filtering algorithms, such as normalized least mean square (NLMS) and affine projection, recursive least squares (RLS), echo removal is achieved by constantly estimating the linear transfer function between the loudspeaker playing the far-end signal and the near-end microphone, known as the echo path. However, in such AEC algorithms, control parameters need to be tuned to ensure fast convergence, and nonlinearity modeling (i.e., nonlinearity introduced by a loudspeaker) is missing.

The inability to model nonlinearity and the need to tune control parameters cast limitations on such adaptive filtering algorithms.

With recent advances in deep neural networks, deep learning-based methods have been utilized for AEC, and their ability to model nonlinear relations leads to promising results, even in challenging noisy or double-talk scenarios. Such methods usually treat AEC as a source separation problem and directly estimate the near-end signal based on the microphone and far-end reference signal. While achieving good performance in general, DNN-based methods have shown limited utility in dealing with continuously changing echo paths.

As an adaptive filtering algorithm for AEC, the frequency domain Kalman filter (FDKF) shows robustness in double-talk scenarios and better convergence rates. Hybrid methods based on the Kalman filter algorithm have been used in research fields such as pose estimation, and speech filtering, but have not been well explored in the domain of AEC. Neural Kalman Filtering, where a DNN is trained to estimate a Kalman gain, omits steps in the Kalman filter and results in a model similar to an NLMS-based hybrid method that does not fully take advantage of the Kalman filter. Hence whether DNN can improve Kalman filter-based AEC remained an open question.

And those reasons there is a desire for technical solutions to such problems that arose in computer audio technology.

SUMMARY

There is included a method and apparatus comprising memory configured to store computer program code and a processor or processors configured to access the computer program code and operate as instructed by the computer program code. The computer program is configured to cause the processor implement receiving code configured to cause

2

the at least one processor to receive an audio signal obtained from a microphone, inputting code configured to cause the at least one processor to input the audio signal into a neural-network based AEC model, wherein the neural-network based AEC model is trained using a training audio signal, and outputting code configured to cause the at least one processor to output an AEC signal from the neural-network based AEC model in which AEC is applied to the audio signal, wherein the AEC signal is a version of the audio signal in which acoustic echo noise of the audio signal is suppressed and target audio of the audio signal is sustained.

According to exemplary embodiments, the neural-network based AEC model comprises a recurrent neural network (RNN) configured to receive an input of the audio signal.

According to exemplary embodiments, the neural-network based AEC model further comprises a first branch and a second branch each configured to, in parallel, receive one or more outputs from the RNN, the first branch estimates a far-end non-linear distortion, the second branch estimates a transition factor, and the second branch further estimates a non-linear transition function.

According to exemplary embodiments, the neural-network based AEC model further comprises a Kalman filter updated based on the far-end non-linear distortion, the transition factor, and the non-linear transition function.

According to exemplary embodiments, the first branch estimates the far-end non-linear distortion by applying a plurality of complex-valued ratio filters (cRF), estimated from a plurality of one-dimensional (1D) convolution layers of the first branch, to the audio signal.

According to exemplary embodiments, the second branch estimates the transition factor by a linear layer followed by a sigmoidal activation function.

According to exemplary embodiments, the second branch estimates the non-linear transition function from a long short-term memory (LSTM) cell comprising 256 hidden units.

According to exemplary embodiments, the RNN comprises a 4-layer LSTM cell of which each layer of the 4-layer LSTM cell comprises 257 hidden units.

According to exemplary embodiments, the neural-network based AEC model further comprises a loss function applied to outputs of both the first branch and the second branch.

According to exemplary embodiments, the neural-network based AHS model is trained with the loss function which comprises a combination of a scale-invariance signal-to-distortion ratio (SI-SDR) in time domain and mean absolute error (MAE) of spectrum magnitude in frequency domain.

BRIEF DESCRIPTION OF THE DRAWINGS

Further features, nature, and various advantages of the disclosed subject matter will be more apparent from the following detailed description and the accompanying drawings in which:

FIG. 1 is a schematic illustration of a diagram in accordance with embodiments;

FIG. 2 is a simplified block diagram in accordance with embodiments;

FIG. 3 is a simplified illustration in accordance with embodiments;

FIG. 4 is a simplified illustration in accordance with embodiments;

3

FIG. 5 is a simplified illustration in accordance with embodiments;

FIG. 6 is a simplified illustration in accordance with embodiments;

FIG. 7 is a simplified illustration in accordance with embodiments;

FIG. 8 is a simplified illustration in accordance with embodiments;

FIG. 9 is a simplified illustration in accordance with embodiments;

FIG. 10 is a simplified flow diagram in accordance with embodiments;

FIG. 11 is a simplified flow diagram in accordance with embodiments; and

FIG. 12 is a simplified illustration in accordance with embodiments.

DETAILED DESCRIPTION

The proposed features discussed below may be used separately or combined in any order. Further, the embodiments may be implemented by processing circuitry (e.g., one or more processors or one or more integrated circuits). In one example, the one or more processors execute a program that is stored in a non-transitory computer-readable medium.

FIG. 1 illustrates a simplified block diagram of a communication system 100 according to an embodiment of the present disclosure. The communication system 100 may include at least two terminals 102 and 103 interconnected via a network 105. For unidirectional transmission of data, a first terminal 103 may code video data at a local location for transmission to the other terminal 102 via the network 105. The second terminal 102 may receive the coded video data of the other terminal from the network 105, decode the coded data and display the recovered video data. Unidirectional data transmission may be common in media serving applications and the like.

FIG. 1 illustrates a second pair of terminals 101 and 104 provided to support bidirectional transmission of coded video that may occur, for example, during videoconferencing. For bidirectional transmission of data, each terminal 101 and 104 may code video data captured at a local location for transmission to the other terminal via the network 105. Each terminal 101 and 104 also may receive the coded video data transmitted by the other terminal, may decode the coded data and may display the recovered video data at a local display device.

In FIG. 1, the terminals 101, 102, 103 and 104 may be illustrated as servers, personal computers and smart phones but the principles of the present disclosure are not so limited. Embodiments of the present disclosure find application with laptop computers, tablet computers, media players and/or dedicated video conferencing equipment. The network 105 represents any number of networks that convey coded video data among the terminals 101, 102, 103 and 104, including for example wireline and/or wireless communication networks. The communication network 105 may exchange data in circuit-switched and/or packet-switched channels. Representative networks include telecommunications networks, local area networks, wide area networks and/or the Internet. For the purposes of the present discussion, the architecture and topology of the network 105 may be immaterial to the operation of the present disclosure unless explained herein below.

FIG. 2 illustrates, as an example for an application for the disclosed subject matter, the placement of a video encoder

4

and decoder in a streaming environment. The disclosed subject matter can be equally applicable to other video enabled applications, including, for example, video conferencing, digital TV, storing of compressed video on digital media including CD, DVD, memory stick and the like, and so on.

A streaming system may include a capture subsystem 203, that can include a video source 201, for example a digital camera, creating, for example, an uncompressed video sample stream 213. That sample stream 213 may be emphasized as a high data volume when compared to encoded video bitstreams and can be processed by an encoder 202 coupled to the video source 201, which may be for example a camera as discussed above. The encoder 202 can include hardware, software, or a combination thereof to enable or implement aspects of the disclosed subject matter as described in more detail below. The encoded video bitstream 204, which may be emphasized as a lower data volume when compared to the sample stream, can be stored on a streaming server 205 for future use. One or more streaming clients 212 and 207 can access the streaming server 205 to retrieve copies 208 and 206 of the encoded video bitstream 204. A client 212 can include a video decoder 211 which decodes the incoming copy of the encoded video bitstream 208 and creates an outgoing video sample stream 210 that can be rendered on a display 209 or other rendering device (not depicted). In some streaming systems, the video bitstreams 204, 206 and 208 can be encoded according to certain video coding/compression standards. Examples of those standards are noted above and described further herein.

FIG. 3 may be a functional block diagram of a video decoder 300 according to an embodiment of the present disclosure.

A receiver 302 may receive one or more codec video sequences to be decoded by the decoder 300; in the same or another embodiment, one coded video sequence at a time, where the decoding of each coded video sequence is independent from other coded video sequences. The coded video sequence may be received from a channel 301, which may be a hardware/software link to a storage device which stores the encoded video data. The receiver 302 may receive the encoded video data with other data, for example, coded audio data and/or ancillary data streams, that may be forwarded to their respective using entities (not depicted). The receiver 302 may separate the coded video sequence from the other data. To combat network jitter, a buffer memory 303 may be coupled in between receiver 302 and entropy decoder/parser 304 ("parser" henceforth). When receiver 302 is receiving data from a store/forward device of sufficient bandwidth and controllability, or from an isosynchronous network, the buffer 303 may not be needed, or can be small. For use on best effort packet networks such as the Internet, the buffer 303 may be required, can be comparatively large and can advantageously of adaptive size.

The video decoder 300 may include a parser 304 to reconstruct symbols 313 from the entropy coded video sequence. Categories of those symbols include information used to manage operation of the decoder 300, and potentially information to control a rendering device such as a display 312 that is not an integral part of the decoder but can be coupled to it. The control information for the rendering device(s) may be in the form of Supplementary Enhancement Information (SEI messages) or Video Usability Information (VUI) parameter set fragments (not depicted). The parser 304 may parse/entropy-decode the coded video sequence received. The coding of the coded video sequence can be in accordance with a video coding technology or

standard, and can follow principles well known to a person skilled in the art, including variable length coding, Huffman coding, arithmetic coding with or without context sensitivity, and so forth. The parser **304** may extract from the coded video sequence, a set of subgroup parameters for at least one of the subgroups of pixels in the video decoder, based upon at least one parameters corresponding to the group. Subgroups can include Groups of Pictures (GOPs), pictures, tiles, slices, macroblocks, Coding Units (CUs), blocks, Transform Units (TUs), Prediction Units (PUs) and so forth. The entropy decoder/parser may also extract from the coded video sequence information such as transform coefficients, quantizer parameter values, motion vectors, and so forth.

The parser **304** may perform entropy decoding/parsing operation on the video sequence received from the buffer **303**, so to create symbols **313**. The parser **304** may receive encoded data, and selectively decode particular symbols **313**. Further, the parser **304** may determine whether the particular symbols **313** are to be provided to a Motion Compensation Prediction unit **306**, a scaler/inverse transform unit **305**, an Intra Prediction Unit **307**, or a loop filter **311**.

Reconstruction of the symbols **313** can involve multiple different units depending on the type of the coded video picture or parts thereof (such as: inter and intra picture, inter and intra block), and other factors. Which units are involved, and how, can be controlled by the subgroup control information that was parsed from the coded video sequence by the parser **304**. The flow of such subgroup control information between the parser **304** and the multiple units below is not depicted for clarity.

Beyond the functional blocks already mentioned, decoder **300** can be conceptually subdivided into a number of functional units as described below. In a practical implementation operating under commercial constraints, many of these units interact closely with each other and can, at least partly, be integrated into each other. However, for the purpose of describing the disclosed subject matter, the conceptual subdivision into the functional units below is appropriate.

A first unit is the scaler/inverse transform unit **305**. The scaler/inverse transform unit **305** receives quantized transform coefficient as well as control information, including which transform to use, block size, quantization factor, quantization scaling matrices, etc. as symbol(s) **313** from the parser **304**. It can output blocks comprising sample values, that can be input into aggregator **310**.

In some cases, the output samples of the scaler/inverse transform **305** can pertain to an intra coded block; that is: a block that is not using predictive information from previously reconstructed pictures, but can use predictive information from previously reconstructed parts of the current picture. Such predictive information can be provided by an intra picture prediction unit **307**. In some cases, the intra picture prediction unit **307** generates a block of the same size and shape of the block under reconstruction, using surrounding already reconstructed information fetched from the current (partly reconstructed) picture **309**. The aggregator **310**, in some cases, adds, on a per sample basis, the prediction information the intra prediction unit **307** has generated to the output sample information as provided by the scaler/inverse transform unit **305**.

In other cases, the output samples of the scaler/inverse transform unit **305** can pertain to an inter coded, and potentially motion compensated block. In such a case, a Motion Compensation Prediction unit **306** can access reference picture memory **308** to fetch samples used for predic-

tion. After motion compensating the fetched samples in accordance with the symbols **313** pertaining to the block, these samples can be added by the aggregator **310** to the output of the scaler/inverse transform unit (in this case called the residual samples or residual signal) so to generate output sample information. The addresses within the reference picture memory form where the motion compensation unit fetches prediction samples can be controlled by motion vectors, available to the motion compensation unit in the form of symbols **313** that can have, for example X, Y, and reference picture components. Motion compensation also can include interpolation of sample values as fetched from the reference picture memory when sub-sample exact motion vectors are in use, motion vector prediction mechanisms, and so forth.

The output samples of the aggregator **310** can be subject to various loop filtering techniques in the loop filter unit **311**. Video compression technologies can include in-loop filter technologies that are controlled by parameters included in the coded video bitstream and made available to the loop filter unit **311** as symbols **313** from the parser **304**, but can also be responsive to meta-information obtained during the decoding of previous (in decoding order) parts of the coded picture or coded video sequence, as well as responsive to previously reconstructed and loop-filtered sample values.

The output of the loop filter unit **311** can be a sample stream that can be output to the render device **312** as well as stored in the reference picture memory **557** for use in future inter-picture prediction.

Certain coded pictures, once fully reconstructed, can be used as reference pictures for future prediction. Once a coded picture is fully reconstructed and the coded picture has been identified as a reference picture (by, for example, parser **304**), the current reference picture **309** can become part of the reference picture buffer **308**, and a fresh current picture memory can be reallocated before commencing the reconstruction of the following coded picture.

The video decoder **300** may perform decoding operations according to a predetermined video compression technology that may be documented in a standard, such as ITU-T Rec. H.265. The coded video sequence may conform to a syntax specified by the video compression technology or standard being used, in the sense that it adheres to the syntax of the video compression technology or standard, as specified in the video compression technology document or standard and specifically in the profiles document therein. Also necessary for compliance can be that the complexity of the coded video sequence is within bounds as defined by the level of the video compression technology or standard. In some cases, levels restrict the maximum picture size, maximum frame rate, maximum reconstruction sample rate (measured in, for example megasamples per second), maximum reference picture size, and so on. Limits set by levels can, in some cases, be further restricted through Hypothetical Reference Decoder (HRD) specifications and metadata for HRD buffer management signaled in the coded video sequence.

In an embodiment, the receiver **302** may receive additional (redundant) data with the encoded video. The additional data may be included as part of the coded video sequence(s). The additional data may be used by the video decoder **300** to properly decode the data and/or to more accurately reconstruct the original video data. Additional data can be in the form of, for example, temporal, spatial, or signal-to-noise ratio (SNR) enhancement layers, redundant slices, redundant pictures, forward error correction codes, and so on.

FIG. 4 may be a functional block diagram of a video encoder 400 according to an embodiment of the present disclosure.

The encoder 400 may receive video samples from a video source 401 (that is not part of the encoder) that may capture 5 video image(s) to be coded by the encoder 400.

The video source 401 may provide the source video sequence to be coded by the encoder (303) in the form of a digital video sample stream that can be of any suitable bit depth (for example: 8 bit, 10 bit, 12 bit, . . .), any colorspace (for example, BT.601 YCrCb, RGB, . . .) and any suitable 10 sampling structure (for example Y CrCb 4:2:0, Y CrCb 4:4:4). In a media serving system, the video source 401 may be a storage device storing previously prepared video. In a videoconferencing system, the video source 401 may be a camera that captures local image information as a video sequence. Video data may be provided as a plurality of individual pictures that impart motion when viewed in sequence. The pictures themselves may be organized as a spatial array of pixels, wherein each pixel can comprise one or more samples depending on the sampling structure, color space, etc. in use. A person skilled in the art can readily understand the relationship between pixels and samples. The description below focuses on samples.

According to an embodiment, the encoder 400 may code 25 and compress the pictures of the source video sequence into a coded video sequence 410 in real time or under any other time constraints as required by the application. Enforcing appropriate coding speed is one function of Controller 402. Controller controls other functional units as described below and is functionally coupled to these units. The coupling is not depicted for clarity. Parameters set by controller can include rate control related parameters (picture skip, quantizer, lambda value of rate-distortion optimization techniques, . . .), picture size, group of pictures (GOP) layout, maximum motion vector search range, and so forth. A person skilled in the art can readily identify other functions of controller 402 as they may pertain to video encoder 400 30 optimized for a certain system design.

Some video encoders operate in what a person skilled in the art readily recognizes as a "coding loop." As an oversimplified description, a coding loop can consist of the encoding part of an encoder 400 ("source coder" henceforth) (responsible for creating symbols based on an input picture to be coded, and a reference picture(s)), and a (local) 45 decoder 406 embedded in the encoder 400 that reconstructs the symbols to create the sample data that a (remote) decoder also would create (as any compression between symbols and coded video bitstream is lossless in the video compression technologies considered in the disclosed subject matter). 50 That reconstructed sample stream is input to the reference picture memory 405. As the decoding of a symbol stream leads to bit-exact results independent of decoder location (local or remote), the reference picture buffer content is also bit exact between local encoder and remote encoder. In other words, the prediction part of an encoder "sees" as reference picture samples exactly the same sample values as a decoder would "see" when using prediction during decoding. This fundamental principle of reference picture synchronicity (and resulting drift, if synchronicity cannot be maintained, for example because of channel errors) is well known to a person skilled in the art.

The operation of the "local" decoder 406 can be the same as of a "remote" decoder 300, which has already been described in detail above in conjunction with FIG. 3. Briefly 65 referring also to FIG. 4, however, as symbols are available and en/decoding of symbols to a coded video sequence by

entropy coder 408 and parser 304 can be lossless, the entropy decoding parts of decoder 300, including channel 301, receiver 302, buffer 303, and parser 304 may not be fully implemented in local decoder 406.

An observation that can be made at this point is that any decoder technology except the parsing/entropy decoding that is present in a decoder also necessarily needs to be present, in substantially identical functional form, in a corresponding encoder. The description of encoder technologies can be abbreviated as they are the inverse of the comprehensively described decoder technologies. Only in certain areas a more detail description is required and provided below.

As part of its operation, the source coder 403 may perform motion compensated predictive coding, which codes an input frame predictively with reference to one or more previously-coded frames from the video sequence that were designated as "reference frames." In this manner, the coding engine 407 codes differences between pixel blocks of an input frame and pixel blocks of reference frame(s) that may be selected as prediction reference(s) to the input frame.

The local video decoder 406 may decode coded video data of frames that may be designated as reference frames, based on symbols created by the source coder 403. Operations of the coding engine 407 may advantageously be lossy processes. When the coded video data may be decoded at a video decoder (not shown in FIG. 4), the reconstructed video sequence typically may be a replica of the source video sequence with some errors. The local video decoder 406 replicates decoding processes that may be performed by the video decoder on reference frames and may cause reconstructed reference frames to be stored in the reference picture memory 405, which may be for example a cache. In this manner, the encoder 400 may store copies of reconstructed reference frames locally that have common content as the reconstructed reference frames that will be obtained by a far-end video decoder (absent transmission errors).

The predictor 404 may perform prediction searches for the coding engine 407. That is, for a new frame to be coded, the predictor 404 may search the reference picture memory 405 for sample data (as candidate reference pixel blocks) or certain metadata such as reference picture motion vectors, block shapes, and so on, that may serve as an appropriate prediction reference for the new pictures. The predictor 404 may operate on a sample block-by-pixel block basis to find appropriate prediction references. In some cases, as determined by search results obtained by the predictor 404, an input picture may have prediction references drawn from multiple reference pictures stored in the reference picture memory 405.

The controller 402 may manage coding operations of the source coder 403, which may be for example a video coder, including, for example, setting of parameters and subgroup parameters used for encoding the video data.

Output of all aforementioned functional units may be subjected to entropy coding in the entropy coder 408. The entropy coder translates the symbols as generated by the various functional units into a coded video sequence, by loss-less compressing the symbols according to technologies known to a person skilled in the art as, for example Huffman coding, variable length coding, arithmetic coding, and so forth.

The transmitter 409 may buffer the coded video sequence(s) as created by the entropy coder 408 to prepare it for transmission via a communication channel 411, which may be a hardware/software link to a storage device which would store the encoded video data. The transmitter 409

may merge coded video data from the source coder **403** with other data to be transmitted, for example, coded audio data and/or ancillary data streams (sources not shown).

The controller **402** may manage operation of the encoder **400**. During coding, the controller **402** may assign to each coded picture a certain coded picture type, which may affect the coding techniques that may be applied to the respective picture. For example, pictures often may be assigned as one of the following frame types:

An Intra Picture (I picture) may be one that may be coded and decoded without using any other frame in the sequence as a source of prediction. Some video codecs allow for different types of Intra pictures, including, for example Independent Decoder Refresh Pictures. A person skilled in the art is aware of those variants of I pictures and their respective applications and features.

A Predictive picture (P picture) may be one that may be coded and decoded using intra prediction or inter prediction using at most one motion vector and reference index to predict the sample values of each block.

A Bi-directionally Predictive Picture (B Picture) may be one that may be coded and decoded using intra prediction or inter prediction using at most two motion vectors and reference indices to predict the sample values of each block. Similarly, multiple-predictive pictures can use more than two reference pictures and associated metadata for the reconstruction of a single block.

Source pictures commonly may be subdivided spatially into a plurality of sample blocks (for example, blocks of 4×4, 8×8, 4×8, or 16×16 samples each) and coded on a block-by-block basis. Blocks may be coded predictively with reference to other (already coded) blocks as determined by the coding assignment applied to the blocks' respective pictures. For example, blocks of I pictures may be coded non-predictively or they may be coded predictively with reference to already coded blocks of the same picture (spatial prediction or intra prediction). Pixel blocks of P pictures may be coded non-predictively, via spatial prediction or via temporal prediction with reference to one previously coded reference pictures. Blocks of B pictures may be coded non-predictively, via spatial prediction or via temporal prediction with reference to one or two previously coded reference pictures.

The encoder **400**, which may be for example a video coder, may perform coding operations according to a pre-determined video coding technology or standard, such as ITU-T Rec. H.265. In its operation, the encoder **400** may perform various compression operations, including predictive coding operations that exploit temporal and spatial redundancies in the input video sequence. The coded video data, therefore, may conform to a syntax specified by the video coding technology or standard being used.

In an embodiment, the transmitter **409** may transmit additional data with the encoded video. The source coder **403** may include such data as part of the coded video sequence. Additional data may comprise temporal/spatial/SNR enhancement layers, other forms of redundant data such as redundant pictures and slices, Supplementary Enhancement Information (SEI) messages, Visual Usability Information (VUI) parameter set fragments, and so on.

FIG. 5 illustrates a simplified block-style workflow diagram **500** of exemplary view-port dependent processing an in Omnidirectional Media Application Format (OMAF) that may allow for 360-degree virtual reality (VR360) streaming described in OMAF.

At acquisition block **501**, video data A is acquired, such as data of multiple images and audio of same time instances

in a case that the image data may represent scenes in VR360. At processing block **503**, the images B_i of the same time instance are processed by one or more of being stitched, mapped onto a projected picture with respect to one or more virtual reality (VR) angles or other angles/viewpoint(s) and region-wise packed. Additionally, metadata may be created indicating any of such processed information and other information so as to assist in delivering and rendering processes.

With respect to data D, at image encoding block **505**, the projected pictures are encoded to data E; and composed into a media file, and in viewport-independent streaming, and at video encoding block **504**, the video pictures are encoded as data E_v as a single-layer bitstream, for example, and with respect to data B_a the audio data may also be encoded into data E_a at audio encoding block **502**.

The data E_a , E_v , and E_i , the entire coded bitstream F_i and/or F may be stored at a (content delivery network (CDN)/cloud) server, and typically may be fully transmitted, such as at delivery block **507** or otherwise, to an OMAF player **520** and may be fully decoded by a decoder such that at least an area of a decoded picture corresponding to a current viewport is rendered to the user at display block **516** with respect to the various metadata, file playback, and orientation/viewport metadata, such as an angle at which a user may be looking through a VR image device with respect to viewport specifications of that device, from the head/eye tracking block **508**. A distinct feature of VR360 is that only a viewport may be displayed at any particular time, and such feature may be utilized to improve the performance of omnidirectional video systems, through selective delivery depending on the user's viewport (or any other criteria, such as recommended viewport timed metadata). For example, viewport-dependent delivery may be enabled by tile-based video coding according to exemplary embodiments.

As with the encoding blocks described above, the OMAF player **520** according to exemplary embodiments may similarly reverse one or more facets of such encoding with respect to the file/segment decapsulation of one or more of the data F' and/or F'_i and metadata, decode the audio data E'_i at audio decoding block **510**, the video data E'_v at video decoding block **513**, and the image data E'_i at image decoding block **514** to proceed with audio rendering of the data B'_a at audio rendering block **511** and image rendering of the data D' at image rendering block **515** so as to output, in a VR360 format according to various metadata such as the orientation/viewport metadata, display data A'_i at display block **516** and audio data A'_s at the loudspeakers/headphones block **512**. The various metadata may influence ones of the data decoding and rendering processes depending on various tracks, languages, qualities, views, that may be selected by or for a user of the OMAF player **520**, and it is to be understood that the order of processing described herein is presented for exemplary embodiments and may be implemented in other orders according to other exemplary embodiments.

FIG. 6 illustrates a simplified block-style content flow process diagram **600** for (coded) point cloud data with view-position and angle dependent processing of point cloud data (herein "V-PCC") with respect to capturing/generating/(de) coding/rendering/displaying 6 degree-of-freedom media. It is to be understood that the described features may be used separately or combined in any order and elements such as for encoding and decoding, among others illustrated, may be implemented by processing circuitry (e.g., one or more processors or one or more integrated circuits), and the

one or more processors may execute a program that is stored in a non-transitory computer-readable medium according to exemplary embodiments.

The diagram 600 illustrates exemplary embodiments for streaming of coded point cloud data according to V-PCC.

At the volumetric data acquisition block 601, a real-world visual scene or a computer-generated visual scene (or combination of them) may be captured by a set of camera devices or synthesized by a computer as a volumetric data, and the volumetric data, which may have an arbitrary format, may be converted to a (quantized) point cloud data format, through image processing at the converting to point cloud block 602. For example, data from the volumetric data may be area data by area data converted into ones of points of the point cloud by pulling one or more of the values described below from the volumetric data and any associated data into a desired point cloud format according to exemplary embodiments. According to exemplary embodiments, the volumetric data may be a 3D data set of 2D images, such as slices from which a 2D projection of the 3D data set may be projected for example. According to exemplary embodiments, point cloud data formats include representations of data points in one or more various spaces and may be used to represent the volumetric data and may offer improvements with respect to sampling and data compression, such as with respect to temporal redundancies, and, for example, a point cloud data in an x, y, z, format representing, at each point of multiple points of the cloud data, color values (e.g., RGB, etc.), luminance, intensity, etc. and could be used with progressive decoding, polygon meshing, direct rendering, octree 3D representations of 2D quadtree data.

At projection to images block 603, the acquired point cloud data may be projected onto 2D images and encoded as image/video pictures with video-based point cloud coding (V-PCC). The projected point cloud data may be composed of attributes, geometry, occupancy map, and other metadata used for point cloud data reconstruction such as with painter's algorithms, ray casting algorithms, (3D) binary space partition algorithms, among others for example.

At the scene generator block 609, on the other hand, a scene generator may generate some metadata to be used for rendering and displaying 6 degrees-of-freedom (DoF) media, by a director's intention or a user's preference for example. Such 6 DoF media may include the 360VR like 3D viewing of a scene from rotational changes on 3D axis X, Y, Z in addition to additional dimension allowing for movement front/back, up/down, and left/right with respect to a virtual experience within or at least according to point cloud coded data. The scene description metadata defines one or more scene composed of the coded point cloud data and other media data, including VR360, light field, audio, etc. and may be provided to one or more cloud servers and or file/segment encapsulation/decapsulation processing as indicated in FIG. 6 and related descriptions.

After video encoding block 604 and image encoding block 605 similar to the video and image encoding described above (and as will be understood, audio encoding also may be provided as described above), file/segment encapsulation block 606 processes such that the coded point cloud data are composed into a media file for file playback or a sequence of an initialization segment and media segments for streaming according to a particular media container file format such as one or more video container formats and such as may be used with respect to DASH, among others as such descriptions represent exemplary embodiments. The file

container also may include the scene description metadata, such as from the scene generator block 1109, into the file or the segments.

According to exemplary embodiments, the file is encapsulated depending on the scene description metadata to include at least one view position and at least one or more angle views at that/those view position(s) each at one or more times among the 6DoF media such that such file may be transmitted on request depending on user or creator input. Further, according to exemplary embodiments, a segment of such file may include one or more portions of such file such as a portion of that 6DoF media indicating a single view-point and angle thereat at one or more times; however, these are merely exemplary embodiments and may be changed depending on various conditions such as network, user, creator capabilities and inputs.

According to exemplary embodiments, the point cloud data is partitioned into multiple 2D/3D regions, which are independently coded such as at one or more of video encoding block 604 and image encoding block 605. Then, each independently coded partition of point cloud data may be encapsulated at file/segment encapsulation block 606 as a track in a file and/or segment. According to exemplary embodiments, each point cloud track and/or a metadata track may include some useful metadata for view-position/angle dependent processing.

According to exemplary embodiments, the metadata, such as included in a file and/or segment encapsulated with respect to the file/segment encapsulation block, useful for the view-position/angle dependent processing includes one or more of the following: layout information of 2D/3D partitions with indices, (dynamic) mapping information associating a 3D volume partition with one or more 2D partitions (e.g. any of a tile/tile group/slice/sub-picture), 3D positions of each 3D partition on a 6DoF coordinate system, representative view position/angle lists, selected view position/angle lists corresponding to a 3D volume partition, indices of 2D/3D partitions corresponding to a selected view position/angle, quality (rank) information of each 2D/3D partition, and rendering information of each 2D/3D partition for example depending on each view position/angle. Calling on such metadata when requested, such as by a user of the V-PCC player or as directed by a content creator for the user of the V-PCC player, may allow for more efficient processing with respect to specific portions of the 6DoF media desired with respect to such metadata such that the V-PCC player may deliver higher quality images of focused on portions of the 6DoF media than other portions rather than delivering unused portions of that media.

From the file/segment encapsulation block 606, the file or one or more segments of the file may be delivered using a delivery mechanism (e.g., by Dynamic Adaptive Streaming over HTTP (DASH)) directly to any of the V-PCC player 625 and a cloud server, such as at the cloud server block 607 at which the cloud server can extract one or more tracks and/or one or more specific 2D/3D partitions from a file and may merge multiple coded point cloud data into one data.

According to data such as with the position/viewing angle tracking block 608, if the current viewing position and angle(s) is/are defined on a 6DoF coordinate system, at a client system, then the view-position/angle metadata may be delivered, from the file/segment encapsulation block 606 or otherwise processed from the file or segments already at the cloud server, at cloud server block 607 such that the cloud sever may extract appropriate partition(s) from the store file(s) and merge them (if necessary) depending on the metadata from the client system having the V-PCC player

13

625 for example, and the extracted data can be delivered to the client, as a file or segments.

With respect to such data, at the file/segment decapsulation block 615, a file decapsulator processes the file or the received segments and extracts the coded bitstreams and parses the metadata, and at video decoding and image decoding blocks 610 and 611, the coded point cloud data are then decoded into decoded and reconstructed, at point cloud reconstruction block 612, to point cloud data, and the reconstructed point cloud data can be displayed at display block 614 and/or may first be composed depending on one or more various scene descriptions at scene composition block 613 with respect to scene description data according to the scene generator block 609.

Embodiments herein may be applied in such environments, such as 2 or more dimensional video conferencing, or hearing aids or karaoke environments or theatre environments or the like that may experience acoustic echo.

For example, embodiments herein integrate a frequency domain Kalman filter (FDKF) and deep neural networks (DNNs) into a hybrid method, herein called KalmanNet, to leverage the advantages of deep learning and adaptive filtering algorithms. Specifically, embodiments employ a DNN to estimate nonlinearly distorted far-end signals, a transition factor, and the nonlinear transition function in the state equation of the FDKF algorithm. Experimental results show that the disclosed KalmanNet embodiments improves the performance of FDKF significantly and outperforms strong baseline methods.

For example, in recent AEC challenges, embodiments herein employ improved two-stage hybrid systems that use DNN as a nonlinear post-processor of a DSP-based adaptive filtering algorithm have shown promising results. In such hybrid systems, DNNs perform nonlinear residual echo suppression, which compensates for the drawbacks of adaptive filtering algorithms. To further leverage the advantages of DNN and adaptive filtering algorithms, methods such as Deep Adaptive AEC may be used to train DNNs where a linear adaptive algorithm is embedded as differentiable layers. Such a hybrid system has shown superior results in modeling a continuously changing echo path compared to DNN-based and two-stage hybrid methods.

A typical acoustic echo scenario with background noise according to embodiments herein is illustrated in example 700 of FIG. 7 and may be understood as in any of the above-described scenarios described with FIG. 1, FIG. 2, FIG. 3, FIG. 4, FIG. 5, FIG. 6, FIG. 7 and other examples noted above. The far-end signal $x(t)$ from far end 703 is transmitted to the near end 701 via a loudspeaker and received by a microphone as acoustic echo $d(t)$:

$$d(t) = h(t) * NL(x(t)) \quad \text{Eq. (1)}$$

where $h(t)$ represents the echo path, $NL(\cdot)$ represents the nonlinear distortion from the loudspeaker, $*$ denotes convolution.

The microphone signal $y(t)$ is composed of echo $d(t)$, near-end speech $s(t)$ and noise $n(t)$:

$$y(t) = s(t) + n(t) + d(t) \quad \text{Eq. (2)}$$

and the $y(t)$ may be processed, with the far-end signal $x(t)$ as a reference, for echo removal before being sent to the

14

far end by the AEC module 702 described further below according to embodiments herein.

An example 800 in FIG. 8 illustrates a simplified block diagram of a frequency-domain Kalman filter for AEC according to embodiments herein to estimate echo signal by modeling the echo path with an adaptive filter $\hat{W}(k)$ where k denotes the frame index. Embodiments herein improve on AEC in clean condition and aim at estimating the near-end speech $s(t)$. FDKF can be interpreted as a two-step procedure (prediction and updating) and the updating of filter weights is achieved through the iterative feedback from the two steps illustrated in FIG. 8 and shown further with the example flowchart 1000 shown in FIG. 10.

In the prediction step, the near-end signal $S(k)$ is estimated at S1001 by the measurement equation,

$$\hat{S}(k) = Y(k) - X(k)\hat{W}(k) \quad \text{Eq. (3)}$$

where $\hat{S}(k)$, $Y(k)$, and $X(k)$ are the short-time Fourier transform (STFT) of the estimated target speech, microphone, and far-end signal respectively. Inverse STFT is applied at S1002 on $\hat{S}(k)$ to obtain the time-domain $\hat{s}(t)$ as well. $\hat{W}(k)$ denotes the estimated echo path in the frequency domain.

In the update step S1003, the state equation for updating echo path $\hat{W}(k)$, shown as element 803 in FIG. 8, is defined as,

$$\hat{W}(k+1) = A[\hat{W}(k) + K(k)\hat{S}(k)] \quad \text{Eq. (4)}$$

where A , shown as element 802 in FIG. 8, is the transition factor. $K(k)$ denotes the Kalman gain.

As shown in FIG. 8, $K(k)$ is related to far-end signal $X(k)$, shown as element 801 in FIG. 8, echo path $\hat{W}(k-1)$ and estimated near-end signal $\hat{S}(k-1)$. The dash line indicates indirect relations. The calculation of $K(k)$ is defined as,

$$K(k) = P(k)X^H(k)[X(k)P(k)X^H(k) + 2\Psi_{vv}(k)]^{-1} \quad \text{Eq. (5)}$$

$$P(k+1) = A^2\left[I - \frac{1}{2}K(k)X(k)\right]P(k) + \Psi_{\Delta\Delta}(k) \quad \text{Eq. (6)}$$

where $P(k)$ is the state estimation error covariance. $\Psi_{vv}(k)$ and $\Psi_{\Delta\Delta}(k)$ are observation noise covariance and process noise covariance respectively and are approximated by the covariance of the estimated near-end signal $\Psi_{ss}(k)$ and the echo-path $\Psi_{\hat{W}\hat{W}}(k)$, respectively.

The time-varying transition factor S1004, the non-linear transition function S1005, and the loss S1006 functions may be applied to the AEC S1007 as a KalmanNet described further below with respect to ones of transition factor A , nonlinear distortion factor $g(\cdot)$, and nonlinear transition function $t(\cdot)$.

For example, FIG. 9 illustrates and example 900 showing a more detailed structure of a KalmanNet according to exemplary embodiments and where $z\text{-}1$ represents a unit delay in the AEC.

As a note shedding light on the technical improvements represented by embodiments herein, while being robust to double-talk and achieving a better convergence rate, the FDKF algorithm still faced several challenges. First, it was realized for embodiments of this disclosure that in the FDKF

algorithm, if the transition factor A in state equation Eq. 4 is set to a constant whose value is manually tuned according to the a of the echo path change variability, then such fixed A would be less likely to adapt well to the changing environment. Second, it was realized for embodiments of this disclosure that if the echo is modeled as a linear transform of far-end signal $X(k)$ then such aspects may neglect the nonlinear distortion caused by amplifiers. Third, it was realized for embodiments of this disclosure that if a linear relationship assumption is used in the updating of Kalman filter weights, such as in the state equation Eq. 4, then it was realized for embodiments of this disclosure that using a nonlinear transition function in the estimation of echo path could help solve nonlinear AEC problems. As such, to address those challenges, there is disclosed a KalmanNet framework and embodiments thereof, where the DNNs are employed to estimate the transition factor A , the far-end nonlinear distortion $g(\cdot)$ and nonlinear transition function $t(\cdot)$ for state equation (Eq. 4).

Viewing the example flowchart **1100** of FIG. **11** in view of example **900** of FIG. **8**, one or more input features may be received at **S1101** and presented to the RNN at **S1102**.

From the RNN at **S1102**, at branch **902**, the transition factor A depicts the variation of the Kalman filter and may have often been manually tuned to a value that is close to 1 in the range of $[0, 1]$, but to incorporate the influence of possible echo path changing to transition factor, rather than using a fixed value, embodiments herein employ DNN to estimate a time-varying transition factor for Eq. 4. That is, it was realized for embodiments of disclosure that since the echo path and nonlinear distortion information can be retrieved from the microphone and far-end signals, in KalmanNet, the input feature computed from the complex STFT of the microphone signal and far-end signal is shared for estimating transition factor A and nonlinear distortion $g(\cdot)$. Similar to NeuralEcho, the employed input feature according to embodiments herein is a concatenation of temporal correlation, frequency correlation, channel covariance, and normalized log power spectrum of microphone and far-end signal. As shown in FIGS. **9** and **11**, an RNN takes the computed input feature at **S1110** and is followed by two branches **901** and **902** for estimating nonlinearly distorted far-end signal $\hat{X}(k)$ and transition factor $\hat{A}(k)$ respectively. The RNN is a 4-layer long short-term memory (LSTM) where each layer has 257 hidden units. The branch for estimating frame-based transition factor $\hat{A}(k)$ at **S1105** is composed of a linear layer at **S1103** followed by a sigmoidal activation function **S1104**.

To address the nonlinear distortion introduced by the loudspeaker, embodiments herein estimate the far-end nonlinear distortion with DNNs and use the nonlinear far-end signal as a reference for updating the Kalman filter. The nonlinearly distorted far-end signal $\hat{X}(k)$ is obtained at **S1109** by applying the complex-valued ratio filters cRF estimated at **S1108** from two one-dimensional convolution layers (Conv. 1D) to the microphone signal $Y(k)$. "Relu" as illustrated refers to a rectified linear unit ("ReLU"). According to exemplary embodiments, the cRF may be applied according to deep filtering: signal extraction and reconstruction using complex timefrequency filters and NeuralEcho features.

To further address the nonlinear AEC problem and incorporate nonlinearity in the estimation of echo path, embodiments herein further employ a modified Eq. 4 by changing the linear transition function to a nonlinear one:

$$\hat{W}(k+1) = t(A[\hat{W}(k) + K(k)\hat{S}(k)]) \quad \text{Eq. (7)}$$

where the nonlinear transition function $t(\cdot)$ is estimated at **S1107** from an LSTM cell at **S1106** which has 256 hidden units. The LSTM cell is trained using $\hat{W}(k)$ from Eq. 4 and previous state $h(k-1)$ as inputs. Then two linear layer takes $h(k)$ as input and outputs the real and imaginary parts of the processed $t(\hat{W}(k))$.

$$\begin{aligned} h_k &= RNN(\hat{W}(k), h_{k-1}) \\ t(\hat{W}(k)) &= FNN(h_k) \end{aligned} \quad \text{Eq. (8)}$$

where RNN and FNN denote the LSTM cell and linear layers respectively.

The loss function at **S1110** is applied to simultaneously optimize scale-invariant signal-to-noise ratio (SI-SDR) in the time domain and mean absolute error (MAE) of magnitude spectrogram between the target and estimated near-end signal as follows:

$$L = -SISDR(s, \hat{s}) + \alpha MAE(|S|, |\hat{S}|) \quad \text{Eq. (9)}$$

where $\hat{S}$ and $\hat{s}$ are the estimated frequency-domain and time-domain near-end signal, respectively. And α according to exemplary embodiments may be set to a value, such as 10,000, to balance the value range of the two losses.

As shown in FIGS. **9** and **11**, the AEC at **S1111** may be applied according to such exemplary embodiments. As such, embodiments herein disclose a learnable Kalman filter for acoustic echo cancellation which advantageously leverages aspects of DNN to improve the Kalman filter by estimating the missing or approximated components, including the transition factor, nonlinear distortion of the far-end signal, and nonlinear transition function for the estimated echo path, and as such, embodiments of this disclosure are effective for echo suppression and outperforms recent baseline methods which were unable to achieve such technical advantages described herein.

According to one or more embodiments of this disclosure, there is improved the frequency-domain Kalman filtering algorithms through deep learning by several aspects. It was determined that simply replacing components in the Kalman filter does not always lead to better performance, but estimating missing or approximated components was discovered to bring improvements. Specifically, embodiments herein utilize DNN to estimate the nonlinearly distorted far-end signal, a transition factor, and the nonlinear transition function in the state equation of the frequency-domain Kalman filter. Experimental results show that modeling the nonlinear distortion in far-end signals yields substantial improvements to the KalmanNet according to one or more embodiments of this disclosure. The transition factor shows adaptations to abrupt echo path changes and introducing a nonlinear transition function in the state equation accelerates training. Compared to modeling the covariance of the state noise and observation noise, it was observed that that injecting a nonlinear transition function in the state equation achieves similar improvement with less computation. The results show that the disclosure hybrid KalmanNet model

embodiments suppress echo well and outperforms attempts at NLMS-based Deep Adaptive AEC.

The techniques described above, can be implemented as computer software using computer-readable instructions and physically stored in one or more computer-readable media or by a specifically configured one or more hardware processors. For example, FIG. 12 shows a computer system 1200 suitable for implementing certain embodiments of the disclosed subject matter.

The computer software can be coded using any suitable machine code or computer language, that may be subject to assembly, compilation, linking, or like mechanisms to create code comprising instructions that can be executed directly, or through interpretation, micro-code execution, and the like, by computer central processing units (CPUs), Graphics Processing Units (GPUs), and the like.

The instructions can be executed on various types of computers or components thereof, including, for example, personal computers, tablet computers, servers, smartphones, gaming devices, internet of things devices, and the like.

The components shown in FIG. 12 for computer system 1200 are exemplary in nature and are not intended to suggest any limitation as to the scope of use or functionality of the computer software implementing embodiments of the present disclosure. Neither should the configuration of components be interpreted as having any dependency or requirement relating to any one or combination of components illustrated in the exemplary embodiment of a computer system 1200.

Computer system 1200 may include certain human interface input devices. Such a human interface input device may be responsive to input by one or more human users through, for example, tactile input (such as: keystrokes, swipes, data glove movements), audio input (such as: voice, clapping), visual input (such as: gestures), olfactory input (not depicted). The human interface devices can also be used to capture certain media not necessarily directly related to conscious input by a human, such as audio (such as: speech, music, ambient sound), images (such as: scanned images, photographic images obtain from a still image camera), video (such as two-dimensional video, three-dimensional video including stereoscopic video).

Input human interface devices may include one or more of (only one of each depicted): keyboard 1201, mouse 1202, trackpad 1203, touch screen 1210, joystick 1205, microphone 1206, scanner 1208, camera 1207.

Computer system 1200 may also include certain human interface output devices. Such human interface output devices may be stimulating the senses of one or more human users through, for example, tactile output, sound, light, and smell/taste. Such human interface output devices may include tactile output devices (for example tactile feedback by the touch-screen 1210, or joystick 1205, but there can also be tactile feedback devices that do not serve as input devices), audio output devices (such as: speakers 1209, headphones (not depicted)), visual output devices (such as screens 1210 to include CRT screens, LCD screens, plasma screens, OLED screens, each with or without touch-screen input capability, each with or without tactile feedback capability-some of which may be capable to output two dimensional visual output or more than three dimensional output through means such as stereographic output; virtual-reality glasses (not depicted), holographic displays and smoke tanks (not depicted)), and printers (not depicted).

Computer system 1200 can also include human accessible storage devices and their associated media such as optical media including CD/DVD ROM/RW 1220 with CD/DVD

1211 or the like media, thumb-drive 1222, removable hard drive or solid state drive 1223, legacy magnetic media such as tape and floppy disc (not depicted), specialized ROM/ASIC/PLD based devices such as security dongles (not depicted), and the like.

Those skilled in the art should also understand that term “computer readable media” as used in connection with the presently disclosed subject matter does not encompass transmission media, carrier waves, or other transitory signals.

Computer system 1200 can also include interface 1299 to one or more communication networks 1298. Networks 1298 can for example be wireless, wireline, optical. Networks 1298 can further be local, wide-area, metropolitan, vehicular and industrial, real-time, delay-tolerant, and so on. Examples of networks 1298 include local area networks such as Ethernet, wireless LANs, cellular networks to include GSM, 3G, 4G, 5G, LTE and the like, TV wireline or wireless wide area digital networks to include cable TV, satellite TV, and terrestrial broadcast TV, vehicular and industrial to include CANBus, and so forth. Certain networks 1298 commonly require external network interface adapters that attached to certain general-purpose data ports or peripheral buses (1250 and 1251) (such as, for example USB ports of the computer system 1200; others are commonly integrated into the core of the computer system 1200 by attachment to a system bus as described below (for example Ethernet interface into a PC computer system or cellular network interface into a smartphone computer system). Using any of these networks 1298, computer system 1200 can communicate with other entities. Such communication can be uni-directional, receive only (for example, broadcast TV), uni-directional send-only (for example CANbusto certain CANbus devices), or bi-directional, for example to other computer systems using local or wide area digital networks. Certain protocols and protocol stacks can be used on each of those networks and network interfaces as described above.

Aforementioned human interface devices, human-accessible storage devices, and network interfaces can be attached to a core 1240 of the computer system 1200.

The core 1240 can include one or more Central Processing Units (CPU) 1241, Graphics Processing Units (GPU) 1242, a graphics adapter 1217, specialized programmable processing units in the form of Field Programmable Gate Areas (FPGA) 1243, hardware accelerators for certain tasks 1244, and so forth. These devices, along with Read-only memory (ROM) 1245, Random-access memory 1246, internal mass storage such as internal non-user accessible hard drives, SSDs, and the like 1247, may be connected through a system bus 1248. In some computer systems, the system bus 1248 can be accessible in the form of one or more physical plugs to enable extensions by additional CPUs, GPU, and the like. The peripheral devices can be attached either directly to the core's system bus 1248, or through a peripheral bus 1249. Architectures for a peripheral bus include PCI, USB, and the like.

CPUs 1241, GPUs 1242, FPGAs 1243, and accelerators 1244 can execute certain instructions that, in combination, can make up the aforementioned computer code. That computer code can be stored in ROM 1245 or RAM 1246. Transitional data can be also be stored in RAM 1246, whereas permanent data can be stored for example, in the internal mass storage 1247. Fast storage and retrieval to any of the memory devices can be enabled through the use of cache memory, that can be closely associated with one or more CPU 1241, GPU 1242, mass storage 1247, ROM 1245, RAM 1246, and the like.

19

The computer readable media can have computer code thereon for performing various computer-implemented operations. The media and computer code can be those specially designed and constructed for the purposes of the present disclosure, or they can be of the kind well known and available to those having skill in the computer software arts.

As an example and not by way of limitation, the computer system having architecture 1200, and specifically the core 1240 can provide functionality as a result of processor(s) (including CPUs, GPUs, FPGA, accelerators, and the like) executing software embodied in one or more tangible, computer-readable media. Such computer-readable media can be media associated with user-accessible mass storage as introduced above, as well as certain storage of the core 1240 that are of non-transitory nature, such as core-internal mass storage 1247 or ROM 1245. The software implementing various embodiments of the present disclosure can be stored in such devices and executed by core 1240. A computer-readable medium can include one or more memory devices or chips, according to particular needs. The software can cause the core 1240 and specifically the processors therein (including CPU, GPU, FPGA, and the like) to execute particular processes or particular parts of particular processes described herein, including defining data structures stored in RAM 1246 and modifying such data structures according to the processes defined by the software. In addition or as an alternative, the computer system can provide functionality as a result of logic hardwired or otherwise embodied in a circuit (for example: accelerator 1244), which can operate in place of or together with software to execute particular processes or particular parts of particular processes described herein. Reference to software can encompass logic, and vice versa, where appropriate. Reference to a computer-readable media can encompass a circuit (such as an integrated circuit (IC)) storing software for execution, a circuit embodying logic for execution, or both, where appropriate. The present disclosure encompasses any suitable combination of hardware and software.

While this disclosure has described several exemplary embodiments, there are alterations, permutations, and various substitute equivalents, which fall within the scope of the disclosure. It will thus be appreciated that those skilled in the art will be able to devise numerous systems and methods which, although not explicitly shown or described herein, embody the principles of the disclosure and are thus within the spirit and scope thereof.

What is claimed is:

1. A method of acoustic echo cancellation (AEC), the method performed by at least one processor and comprising: receiving an audio signal obtained from a microphone; inputting the audio signal into a neural-network based AEC model, wherein the neural-network based AEC model is trained using a training audio signal; and outputting an AEC signal from the neural-network based AEC model in which AEC is applied to the audio signal, wherein the AEC signal is a version of the audio signal in which acoustic echo noise of the audio signal is suppressed and target audio of the audio signal is sustained, and wherein the neural-network based AEC model outputs the AEC signal based on estimating a far-end non-linear distortion, a transition factor, and a non-linear transition function.
2. The method according to claim 1, wherein the neural-network based AEC model comprises a recurrent neural network (RNN) configured to receive an input of the audio signal.

20

3. The method according to claim 2, wherein the neural-network based AEC model further comprises a first branch and a second branch each configured to, in parallel, receive one or more outputs from the RNN, wherein the first branch estimates the far-end non-linear distortion, wherein the second branch estimates the transition factor, and wherein the second branch further estimates the non-linear transition function.
4. The method according to claim 3, wherein the neural-network based AEC model further comprises a Kalman filter updated based on the far-end non-linear distortion, the transition factor, and the non-linear transition function.
5. The method according to claim 4, wherein the first branch estimates the far-end non-linear distortion by applying a plurality of complex-valued ratio filters (cRF), estimated from a plurality of one-dimensional (1D) convolution layers of the first branch, to the audio signal.
6. The method according to claim 4, wherein the second branch estimates the transition factor by a linear layer followed by a sigmoidal activation function.
7. The method according to claim 6, wherein the second branch estimates the non-linear transition function from a long short-term memory (LSTM) cell comprising 256 hidden units.
8. The method according to claim 7, wherein the RNN comprises a 4-layer LSTM cell of which each layer of the 4-layer LSTM cell comprises 257 hidden units.
9. The method according to claim 4, wherein the neural-network based AEC model further comprises a loss function applied to outputs of both the first branch and the second branch.
10. The method according to claim 9, wherein the neural-network based AEC model is trained with the loss function which comprises a combination of a scale-invariance signal-to-distortion ratio (SISDR) in time domain and mean absolute error (MAE) of spectrum magnitude in frequency domain.
11. An apparatus for acoustic echo cancellation (AEC), the apparatus comprising:
 - at least one memory configured to store computer program code;
 - at least one processor configured to access the computer program code and operate as instructed by the computer program code, the computer program code including:
 - receiving code configured to cause the at least one processor to receive an audio signal obtained from a microphone;
 - inputting code configured to cause the at least one processor to input the audio signal into a neural-network based AEC model, wherein the neural-network based AEC model is trained using a training audio signal; and
 - outputting code configured to cause the at least one processor to output an AEC signal from the neural-network based AEC model in which AEC is applied to the audio signal, wherein the AEC signal is a version of the audio signal in which acoustic echo noise of the audio signal is suppressed and target audio of the audio signal is sustained, and wherein the neural-network based AEC model outputs the AEC signal based on

21

estimating a far-end non-linear distortion, a transition factor, and a non-linear transition function.

12. The apparatus according to claim 11,

wherein the neural-network based AEC model comprises a recurrent neural network (RNN) configured to receive an input of the audio signal.

13. The apparatus according to claim 12,

wherein the neural-network based AEC model further comprises a first branch and a second branch each configured to, in parallel, receive one or more outputs from the RNN,

wherein the first branch estimates the far-end non-linear distortion,

wherein the second branch estimates the transition factor, and

wherein the second branch further estimates the non-linear transition function.

14. The apparatus according to claim 13,

wherein the neural-network based AEC model further comprises a Kalman filter updated based on the far-end non-linear distortion, the transition factor, and the non-linear transition function.

15. The apparatus according to claim 14,

wherein the first branch estimates the far-end non-linear distortion by applying a plurality of complex-valued ratio filters (cRF), estimated from a plurality of one-dimensional (1D) convolution layers of the first branch, to the audio signal.

22

16. The apparatus according to claim 14,

wherein the second branch estimates the transition factor by a linear layer followed by a sigmoidal activation function.

17. The apparatus according to claim 16,

wherein the second branch estimates the non-linear transition function from a long short-term memory (LSTM) cell comprising 256 hidden units.

18. The apparatus according to claim 17,

wherein the RNN comprises a 4-layer LSTM cell of which each layer of the 4-layer LSTM cell comprises 257 hidden units.

19. The apparatus according to claim 14,

wherein the neural-network based AEC model further comprises a loss function applied to outputs of both the first branch and the second branch.

20. A non-transitory computer readable medium storing a program causing a computer to:

receive an audio signal obtained from a microphone;

input the audio signal into a neural-network based AEC model, wherein the neural-network based AEC model is trained using a training audio signal; and

output an AEC signal from the neural-network based AEC model in which AEC is applied to the audio signal, wherein the AEC signal is a version of the audio signal in which acoustic echo noise of the audio signal is suppressed and target audio of the audio signal is sustained, and wherein the neural-network based AEC model outputs the AEC signal based on estimating a far-end non-linear distortion, a transition factor, and a non-linear transition function.

* * * * *